



重慶理工大學

深度学习

学 院 计算机科学与工程学院

学 号

姓 名

指 导 教 师

时 间

实验 2 环境搭建与 pytorch 初步应用

实验目的:

- 1) 熟悉 python 编程运用
- 2) 学习 PyTorch 基本编程

实验内容:

Ex1 自行上网查阅相关资料,学习在 Anaconda 下安装库,环境中安装 Python (3.7.5)、Pytorch 安装等。

Ex2 检验安装 torch 成功

Ex3 实现 torch 向量处理的三个代码。(自由发挥)

Ex1 环境搭建说明:

在仓库根目录使用 Miniconda 创建 dl 环境 (Python 3.7.5), 并安装 PyTorch。主要步骤如下:

1. 使用 `environment.yml` 创建 conda 环境;
2. 激活环境后执行 `pip install torch==1.13.1` 安装 PyTorch;
3. 使用 `python -c "import torch"` 验证安装。

Ex2 源代码:

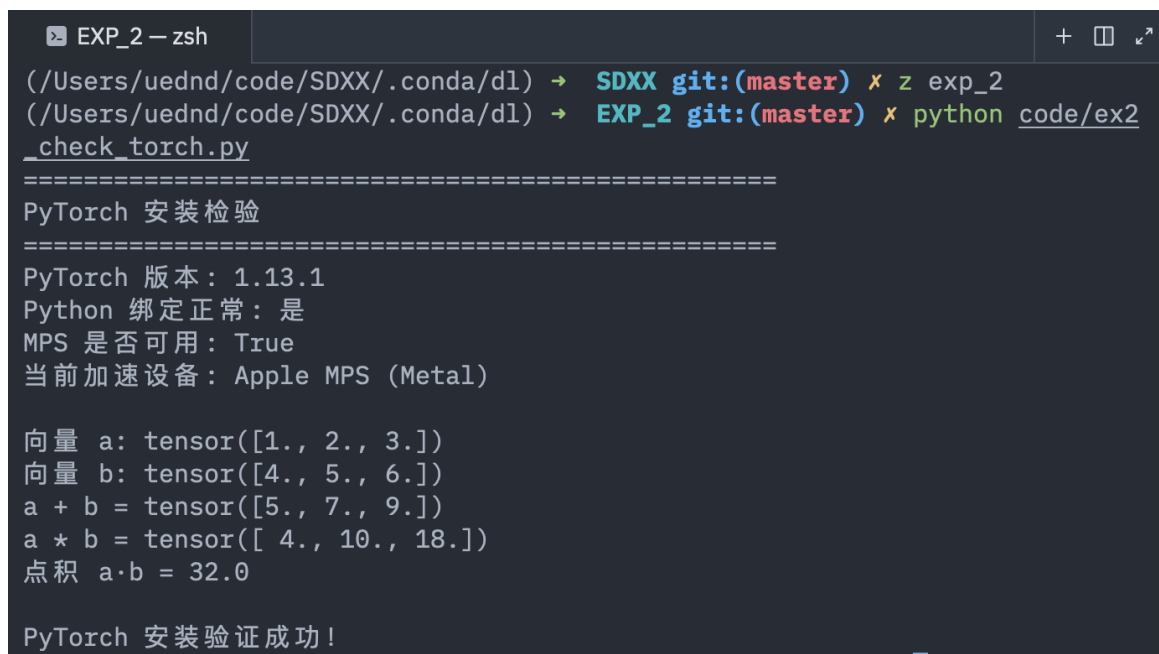
```
import torch
```

```
print("=" * 50)
print("PyTorch 安装检验")
print("=" * 50)
print("PyTorch 版本:", torch.__version__)
print("Python 绑定正常: 是")
mps_available = hasattr(torch.backends, "mps") and torch.backends.mps.is_available()
print("MPS 是否可用:", mps_available)
if mps_available:
    print("当前加速设备: Apple MPS (Metal)")
else:
    print("当前设备: CPU")
```

```
# 基本张量运算验证
a = torch.tensor([1.0, 2.0, 3.0])
b = torch.tensor([4.0, 5.0, 6.0])
print("\n向量 a:", a)
print("向量 b:", b)
```

```
print("a + b =", a + b)
print("a * b =", a * b)
print("点积 a · b =", torch.dot(a, b).item())
print("\nPyTorch 安装验证成功!")
```

运行（测试）过程及结果：



```
EXP_2 - zsh
(/Users/uednd/code/SDXX/.conda/dl) → SDXX git:(master) x z exp_2
(/Users/uednd/code/SDXX/.conda/dl) → EXP_2 git:(master) x python code/ex2
_check_torch.py
=====
PyTorch 安装检验
=====
PyTorch 版本：1.13.1
Python 绑定正常：是
MPS 是否可用：True
当前加速设备：Apple MPS (Metal)

向量 a: tensor([1., 2., 3.])
向量 b: tensor([4., 5., 6.])
a + b = tensor([5., 7., 9.])
a * b = tensor([ 4., 10., 18.])
点积 a·b = 32.0

PyTorch 安装验证成功!
```

Ex3 源代码 1:

```
import torch
```

```
print("=" * 50)
print("张量创建与形状变换")
print("=" * 50)
```

```
# 1. 从不同数据源创建张量
```

```
lst = [1, 2, 3, 4]
t1 = torch.tensor(lst, dtype=torch.float32)
t2 = torch.zeros(2, 2)
t3 = torch.ones(3)
t4 = torch.arange(0, 10, 2)
```

```
print("从列表创建:", t1)
print("zeros(2,2) :\n", t2)
print("ones(3):", t3)
print("arange(0,10,2) :", t4)
```

```
# 2. 形状变换与索引
```

```
x = torch.arange(12).reshape(3, 4)
```

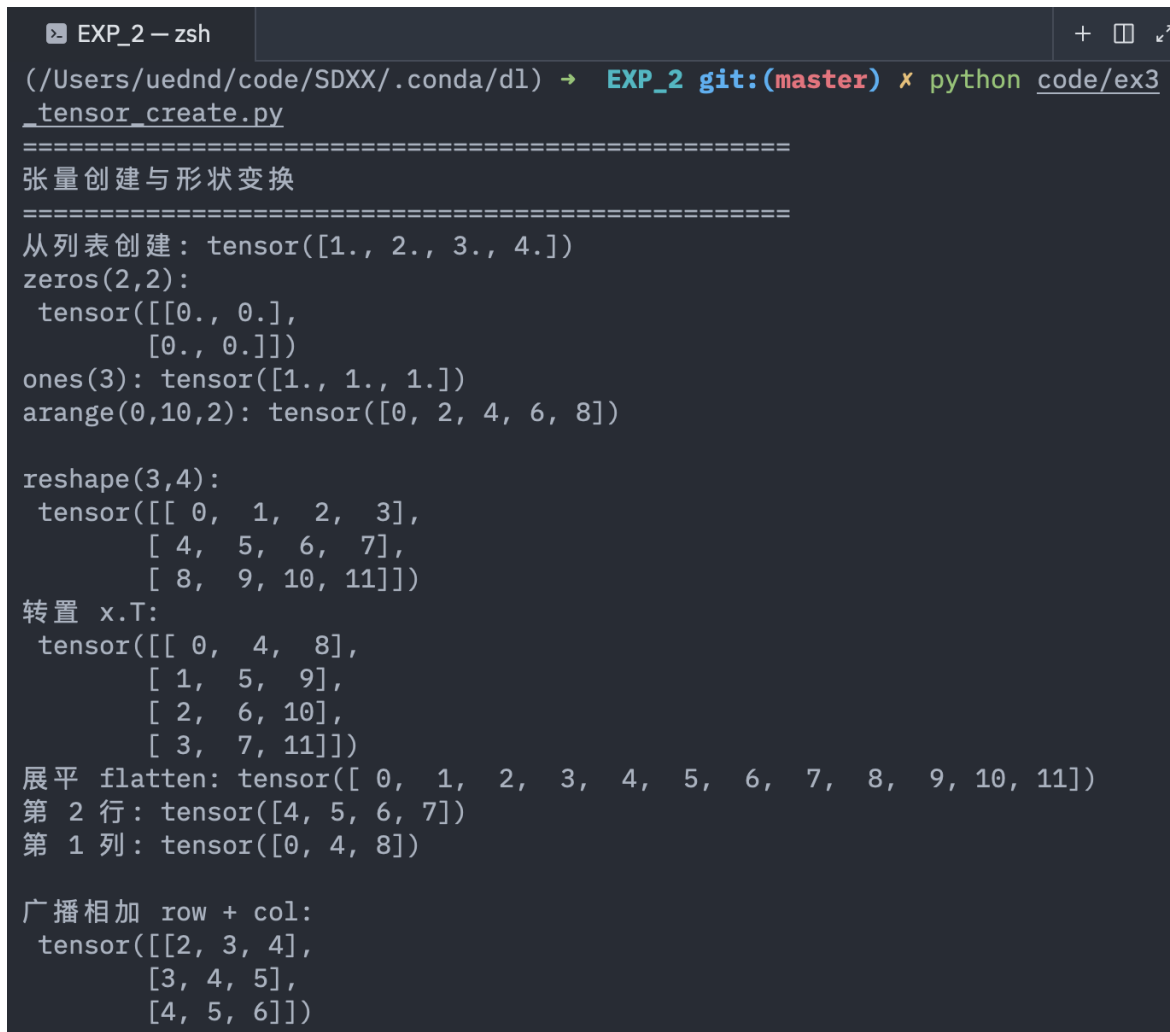
```

print("\nreshape(3,4):\n", x)
print("转置 x.T:\n", x.T)
print("展平 flatten:", x.flatten())
print("第 2 行:", x[1])
print("第 1 列:", x[:, 0])

# 3. 广播机制
row = torch.tensor([[1, 2, 3]])
col = torch.tensor([[1], [2], [3]])
print("\n广播相加 row + col:\n", row + col)

```

运行（测试）过程及结果：



```

EXP_2 - zsh
(/Users/uednd/code/SDXX/.conda/dl) → EXP_2 git:(master) x python code/ex3_tensor_create.py
=====
张量创建与形状变换
=====
从列表创建: tensor([1., 2., 3., 4.])
zeros(2,2):
  tensor([[0., 0.],
          [0., 0.]])
ones(3): tensor([1., 1., 1.])
arange(0,10,2): tensor([0, 2, 4, 6, 8])

reshape(3,4):
  tensor([[ 0,  1,  2,  3],
          [ 4,  5,  6,  7],
          [ 8,  9, 10, 11]])
转置 x.T:
  tensor([[ 0,  4,  8],
          [ 1,  5,  9],
          [ 2,  6, 10],
          [ 3,  7, 11]])
展平 flatten: tensor([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
第 2 行: tensor([4, 5, 6, 7])
第 1 列: tensor([0, 4, 8])

广播相加 row + col:
  tensor([[2, 3, 4],
          [3, 4, 5],
          [4, 5, 6]])

```

Ex3 源代码 2:

```

import time

import torch

```

```

def benchmark_mm(a, b, device):
    """在指定设备上执行矩阵乘法并返回耗时（秒）。"""
    if device == "mps":
        a_dev = a.to("mps")
        b_dev = b.to("mps")
    else:
        a_dev, b_dev = a, b

    # 预热，避免首次调用开销影响计时
    warmup = torch.mm(a_dev, b_dev)
    if device == "mps":
        warmup.cpu()

    start = time.time()
    result = torch.mm(a_dev, b_dev)
    if device == "mps":
        result.cpu()
    return time.time() - start

print("=" * 50)
print("张量运算与 CPU/MPS 对比")
print("=" * 50)

# 1. 向量运算
v1 = torch.randn(5)
v2 = torch.randn(5)
print("随机向量 v1:", v1)
print("随机向量 v2:", v2)
print("向量范数 ||v1||:", torch.norm(v1).item())
print("点积 v1 · v2:", torch.dot(v1, v2).item())

# 2. 矩阵乘法
A = torch.tensor([[1.0, 2.0], [3.0, 4.0]])
B = torch.tensor([[5.0, 6.0], [7.0, 8.0]])
C = torch.mm(A, B)
print("\n矩阵 A:\n", A)
print("矩阵 B:\n", B)
print("A @ B:\n", C)

# 3. CPU 与 MPS 运算耗时对比
size = 2000
a = torch.randn(size, size)
b = torch.randn(size, size)

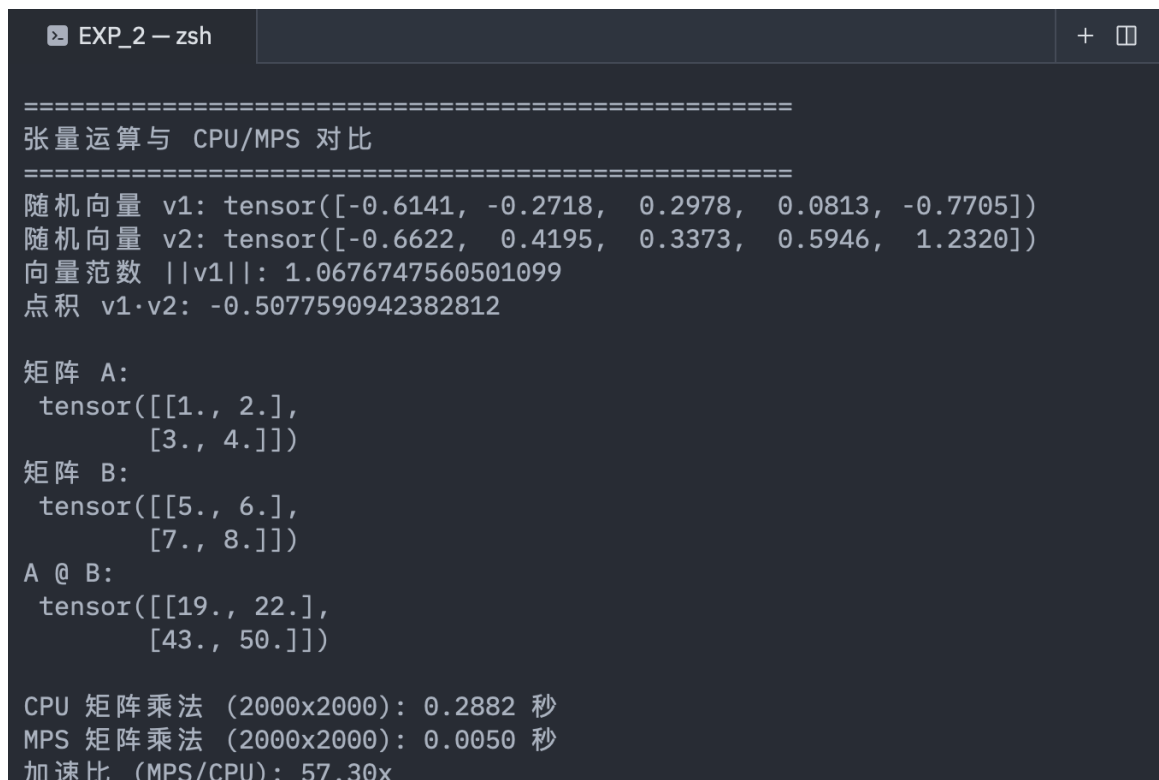
cpu_time = benchmark_mm(a, b, "cpu")
print(f"\nCPU 矩阵乘法 ({size}x{size}): {cpu_time:.4f} 秒")

if hasattr(torch.backends, "mps") and torch.backends.mps.is_available():

```

```
mps_time = benchmark_mm(a, b, "mps")
print(f"MPS 矩阵乘法 ({size}x{size}): {mps_time:.4f} 秒")
print(f"加速比 (MPS/CPU): {cpu_time / mps_time:.2f}x")
print("说明: MPS 是 Apple Silicon 上的 GPU 加速后端。")
else:
    print("本机未启用 MPS, 仅完成 CPU 测试。")
```

运行（测试）过程及结果：



```
EXP_2 - zsh
=====
张量运算与 CPU/MPS 对比
=====
随机向量 v1: tensor([-0.6141, -0.2718,  0.2978,  0.0813, -0.7705])
随机向量 v2: tensor([-0.6622,  0.4195,  0.3373,  0.5946,  1.2320])
向量范数 ||v1||: 1.0676747560501099
点积 v1·v2: -0.5077590942382812

矩阵 A:
  tensor([[1., 2.],
          [3., 4.]])
矩阵 B:
  tensor([[5., 6.],
          [7., 8.]])
A @ B:
  tensor([[19., 22.],
          [43., 50.]])

CPU 矩阵乘法 (2000x2000): 0.2882 秒
MPS 矩阵乘法 (2000x2000): 0.0050 秒
加速比 (MPS/CPU): 57.30x
```

Ex3 源代码 3：

```
import torch

print("=" * 50)
print("张量统计与维度聚合")
print("=" * 50)

data = torch.tensor ([[1.0, 2.0, 3.0],
                      [4.0, 5.0, 6.0]])
print("原始张量:\n", data)

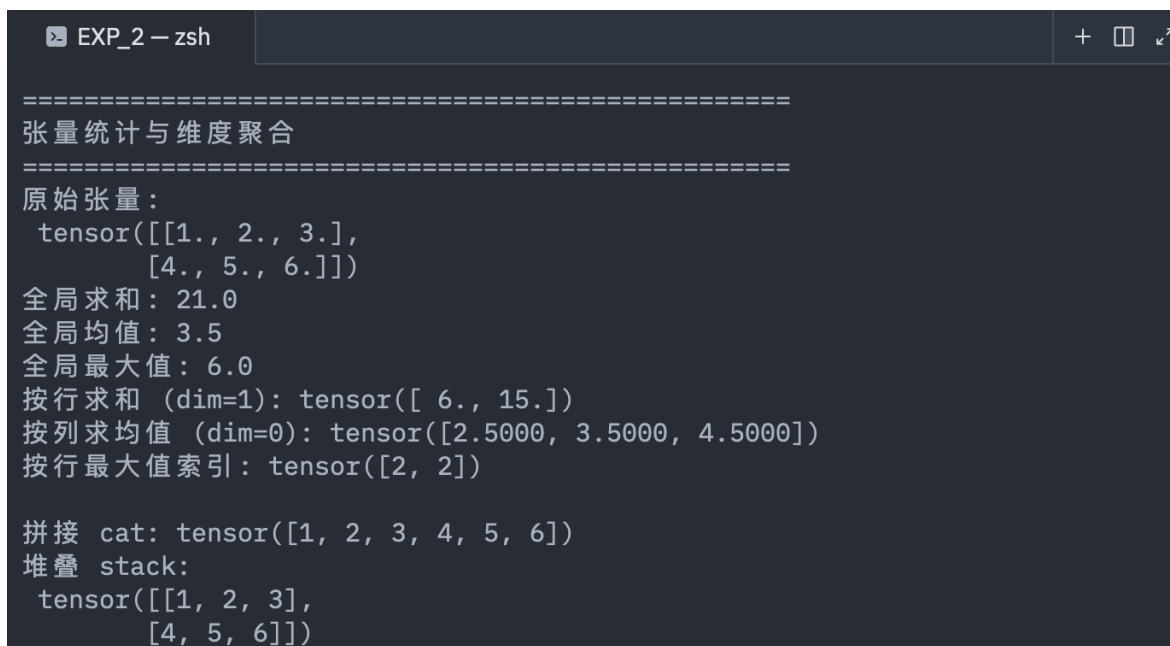
print("全局求和:", data.sum().item())
print("全局均值:", data.mean().item())
print("全局最大值:", data.max().item())
```

```
print("按行求和 (dim=1):", data.sum(dim=1))
print("按列求均值 (dim=0):", data.mean(dim=0))
print("按行最大值索引:", data.argmax(dim=1))
```

拼接与堆叠

```
a = torch.tensor([1, 2, 3])
b = torch.tensor([4, 5, 6])
print("\n拼接 cat:", torch.cat([a, b]))
print("堆叠 stack:\n", torch.stack([a, b]))
```

运行（测试）过程及结果：



```
EXP_2 - zsh
=====
张量统计与维度聚合
=====
原始张量：
  tensor([[1., 2., 3.],
          [4., 5., 6.]])
全局求和：21.0
全局均值：3.5
全局最大值：6.0
按行求和 (dim=1): tensor([ 6., 15.])
按列求均值 (dim=0): tensor([2.5000, 3.5000, 4.5000])
按行最大值索引: tensor([2, 2])

拼接 cat: tensor([1, 2, 3, 4, 5, 6])
堆叠 stack:
  tensor([[1, 2, 3],
          [4, 5, 6]])
```

总结:

建立深度学习框架的基础认知，理解张量作为 PyTorch 核心数据结构的灵活性与高效性。

通过对比 CPU 与 MPS 设备的张量运算，意识到硬件加速对深度学习的重要性。

本次实验完成了以下工作：

1. 在 Anaconda dl 环境中成功安装 PyTorch 1.13.1，并通过 import 与基本向量运算验证安装正确；
2. 掌握了张量的创建方式 (tensor、zeros、ones、arange) 及形状变换 (reshape、transpose、flatten)；
3. 实践了向量点积、矩阵乘法等核心运算，并对比了 CPU 与 MPS 上大规模矩阵乘法的耗时差异。